

Chapitre 1 : Gérer les erreurs, un programme qui ne plante pas

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python avancé · **Fil rouge** : le registre des colis de Yonnee Express · **Version** : 2026-10-18

Fiche du chapitre

Objectifs	A la fin du chapitre, l'apprenant sait : 1) lire un message d'erreur Python (le *traceback*) et trouver la ligne en cause ; 2) distinguer erreur de syntaxe et exception, et reconnaître les exceptions courantes ; 3) intercepter une exception avec `try` / `except`, en visant la bonne ; 4) utiliser `else` et `finally` ; 5) redemander une saisie tant qu'elle est invalide ; 6) lever une exception avec `raise` et comprendre comment elle remonte les appels ; 7) utiliser la hiérarchie des exceptions et créer une exception personnalisée.
Prérequis	Python de L1 : variables, conditions, boucles, listes, dictionnaires, fonctions
Durée	🟢 Débutant : 8 h · 🔵 Standard : 6 h · 🟣 Expert : 4 h (atelier compris)
Matériel	Python 3.12 ou plus récent (le cours a été vérifié avec Python 3.13), VS Code avec l'extension Python, Git, un compte GitHub
Livrable	L'atelier 1 : la saisie fiable des livraisons, publiée sur GitHub

Comment lire ce chapitre

- **Noyau** : obligatoire pour tous les niveaux.
- 🟢 **Débutant** : explications pas à pas. A sauter si le public est à l'aise.
- 🔵 **Standard** : le niveau de référence.
- 🟣 **Expert** : approfondissements.

Chaque morceau de code nouveau est décortiqué dans un tableau « Morceau / Ce qu'il veut dire ». **Chaque exemple est à taper dans un fichier et à lancer** ; les encadrés 🖍️ **Essayez** vous font provoquer les erreurs. Créez pour ce chapitre un dossier `essais-ch1`, ouvrez-le dans VS Code, et lancez chaque fichier dans le terminal avec `python nom_du_fichier.py` (ou `py nom_du_fichier.py` sous Windows).

Le plan du cours

Ch	Thème	Ce que le fil rouge devient
1	**Gérer les erreurs**	La saisie d'une livraison ne plante plus, quelle que soit la réponse de l'utilisateur
2	Les fichiers (texte, CSV, JSON)	Le registre est enregistré et rechargé, les lignes abîmées sont signalées
3	Structures de données avancées	Des statistiques sur les livreurs, les quartiers, les retards
4	Fonctions avancées	Des rapports et des filtres à la demande
5	Générateurs et décorateurs	Un registre de 500 000 lignes traité sans saturer la mémoire
6	Organiser un programme	L'outil complet, en plusieurs modules, avec un journal d'exécution
7	Introduction à la programmation orientée objet	Le registre réécrit avec des classes

Le **fil rouge** est le registre des livraisons de **Yonnee Express**, un service de livraison de colis à Dakar (une entreprise imaginaire ; *yónnee* veut dire « envoyer » en wolof).

---

1. Pourquoi gérer les erreurs ?

En L1, quand votre programme recevait une réponse inattendue, il s'arrêtait avec un message rouge. Pour un exercice, ce n'est pas grave. Pour le livreur de Yonnee Express qui saisit sa vingtième livraison de la journée, c'est une catastrophe : une faute de frappe, et tout ce qu'il a saisi est perdu.

Un programme professionnel suit une règle simple : **il ne doit jamais s'arrêter brutalement à cause d'une saisie ou d'un fichier**. Il doit prévoir les problèmes, les expliquer à l'utilisateur, et continuer. C'est l'objet de ce chapitre.

Il y a deux grandes familles d'erreurs :

Famille	Quand	Exemple	Peut-on l'intercepter ?
Erreur de syntaxe	**Avant** l'exécution : Python lit le fichier et ne le comprend pas	Un `:` oublié après un `if`	Non : il faut corriger le code
Exception	**Pendant** l'exécution : le code est correct, mais une situation imprévue arrive	L'utilisateur tape « vingt » au lieu de 20	Oui : c'est tout le sujet du chapitre

2. Lire un message d'erreur

2.1 Le *traceback*

Créez le fichier `traceback_demo.py` :

```python
def moyenne(montants):
    return sum(montants) / len(montants)


def rapport(livreur, montants):
    m = moyenne(montants)
    print(f"{livreur} : moyenne {m:.0f} F")


rapport("Modou", [2500, 3000, 1500])
rapport("Astou", [])
print("Fin du rapport")
```

Lancez-le avec `python traceback_demo.py`. Résultat (avec Python 3.13 ; le chemin complet de votre fichier apparaît à la place de `traceback_demo.py`) :

```
Modou : moyenne 2333 F
Traceback (most recent call last):
  File "traceback_demo.py", line 11, in <module>
    rapport("Astou", [])
    ~~~~~~~^^^^^^^^^^^^^
  File "traceback_demo.py", line 6, in rapport
    m = moyenne(montants)
  File "traceback_demo.py", line 2, in moyenne
    return sum(montants) / len(montants)
           ~~~~~~~~~~~~~~~^~~~~~~~~~~~~~
ZeroDivisionError: division by zero
```

Ce bloc s'appelle un **traceback** (« trace de retour ») : c'est le chemin suivi par le programme jusqu'à l'erreur.

Morceau	Ce qu'il veut dire
`Modou : moyenne 2333 F`	Le premier rapport a fonctionné : tout ce qui est **avant** l'erreur s'est exécuté
`Traceback (most recent call last):`	« Trace, l'appel le plus récent en dernier » : on lit les appels du premier (en haut) au dernier (en bas)
`File "traceback_demo.py", line 11, in <module>`	Le fichier, la ligne (11) et l'endroit : `<module>` veut dire « le programme principal », hors de toute fonction
`File "...", line 6, in rapport`	Ligne 11, le programme a appelé `rapport` ; dans `rapport`, ligne 6, on a appelé `moyenne`
`line 2, in moyenne`	C'est **ici** que l'erreur s'est produite : la dernière ligne du chemin
`~~~~~~~~~~~~~~~^~~~~~~~~~~~~~`	Python souligne l'expression en cause ; le `^` montre l'opération : la division `/`
`ZeroDivisionError: division by zero`	La **dernière ligne** : le **type** de l'exception, puis son **message**. `len([])` vaut 0 : division par zéro

Morceau	Ce qu'il veut dire
`Fin du rapport` n'apparaît pas	Le programme s'est **arrêté** à l'erreur : rien de ce qui suit n'a été exécuté

> 🟢 **Débutant : la méthode de lecture en trois temps**
>
> 1. Lisez la **dernière ligne** : le type et le message disent **quoi**. 2) Lisez la ligne `File ...` juste au-dessus : elle dit **où**. 3) Remontez les lignes `File ...` pour comprendre **par quel chemin** on est arrivé là. Ici : quoi = division par zéro ; où = ligne 2, dans `moyenne` ; chemin = `rapport("Astou", [])` a donné une liste vide.

2.2 Les erreurs de syntaxe

Créez `e_syntax.py`, avec un `:` oublié :

```
if montant > 1000
    print("cher")
```

```
  File "e_syntax.py", line 1
    if montant > 1000
                      ^
SyntaxError: expected ':'
```

Il n'y a pas de ligne `Traceback` : le programme n'a **pas démarré du tout**. Python a lu le fichier, n'a pas compris la ligne 1, et dit ce qu'il attendait (`expected ':'`, « un `:` était attendu »).

> 🧨 **Essayez** : dans un fichier `e_indent.py`, écrivez une fonction dont le corps n'est pas décalé :
>
> ```
> def frais(poids):
> return poids * 100
> ```
>
> Message : `IndentationError: expected an indented block after function definition on line 1` (« un bloc décalé était attendu après la définition de fonction de la ligne 1 »). En Python, le décalage **fait partie de la syntaxe**.

2.3 Les exceptions que vous rencontrerez le plus

Créez un fichier par ligne du tableau, lancez-le, et comparez avec le message affiché. Tous les messages ont été obtenus avec Python 3.13.

Code	Exception et message	Ce qu'elle veut dire
`age = int("vingt")`	`ValueError: invalid literal for int() with base 10: 'vingt'`	**Valeur** incorrecte : le type est bon (un texte), mais son contenu ne peut pas devenir un entier
`total = "Montant : " + 2500`	`TypeError: can only concatenate str (not "int") to str`	**Type** incorrect : on ne peut coller (concaténer) qu'un texte à un texte. Il faut `str(2500)` ou une f-string
`print(10 / 0)`	`ZeroDivisionError: division by zero`	Division par zéro
`quartiers = ["Médina", "Plateau", "Yoff"]` puis `quartiers[3]`	`IndexError: list index out of range`	Indice hors de la liste : les indices vont de 0 à 2
`tarifs = {"Médina": 1000, "Plateau": 1500}` puis `tarifs["Yoff"]`	`KeyError: 'Yoff'`	Clé absente du dictionnaire
`montant = 2500` puis `print(montnat)`	`NameError: name 'montnat' is not defined. Did you mean: 'montant'?`	Nom inconnu (souvent une faute de frappe). Python propose même la correction
`nom = "modou"` puis `nom.uper()`	`AttributeError: 'str' object has no attribute 'uper'. Did you mean: 'upper'?`	La méthode (ou l'attribut) n'existe pas pour ce type
`open("livraisons_2026.csv")`	`FileNotFoundError: [Errno 2] No such file or directory: 'livraisons_2026.csv'`	Fichier introuvable (chapitre 2)
`import panda`	`ModuleNotFoundError: No module named 'panda'`	Module introuvable : faute de frappe (`pandas`) ou module non installé

🔵 **Standard : les versions de Python** Les suggestions `Did you mean: ...?` et les soulignements `~~~^~~~` sont apparus avec les versions récentes (3.10 à 3.13). Avec une version plus ancienne, le **type** et le **message** principal sont les mêmes, mais l'affichage est moins détaillé. Vérifiez votre version avec `python --version` .

3. Intercepter une exception : `try` / `except`

3.1 La forme de base

Créez `essai1.py` :

```
texte = input("Montant du colis (F) : ")
try:
    montant = int(texte)
    print("Montant enregistré :", montant)
except ValueError:
    print("Ce n'est pas un nombre entier :", texte)
print("Suite du programme")
```

Lancez-le deux fois : tapez d'abord `2500`, puis `deux mille`.

```
Montant du colis (F) : 2500
Montant enregistré : 2500
Suite du programme
```

```
Montant du colis (F) : deux mille
Ce n'est pas un nombre entier : deux mille
Suite du programme
```

Morceau	Ce qu'il veut dire
`try:`	« essaie » : Python exécute le bloc décalé qui suit, en **surveillant** les exceptions
`montant = int(texte)`	La ligne qui peut échouer. Si elle échoue, Python **saute immédiatement** au `except` : la ligne `print("Montant enregistré ...")` n'est pas exécutée
`except ValueError:`	« en cas de `ValueError` » : le bloc exécuté si **cette** exception se produit dans le `try`
`print("Suite du programme")`	Dans les deux cas, le programme **continue** après le `try` / `except` : il n'a pas planté

Avec `2500`, le bloc `except` est ignoré. Avec `deux mille`, `int()` lève une `ValueError`, que le `except` **intercepte** (on dit aussi « attrape »).

> 🖍️ **Essayez** : remplacez `except ValueError:` par `except ZeroDivisionError:` et tapez `deux mille`. Le programme plante avec le traceback de la `ValueError` : un `except` n'attrape **que** le type d'exception qu'il nomme. Annulez.

3.2 Plusieurs `except`, et le message de l'exception

Créez `essai2.py` :

```python
def prix_au_kilo(montant, poids):
    return montant / poids


saisies = [("2500", "2"), ("3000", "0"), ("abc", "1"), ("1800", "")]
for m, p in saisies:
    try:
        resultat = prix_au_kilo(int(m), float(p))
        print(f"{m} F pour {p} kg : {resultat:.0f} F/kg")
    except ValueError as erreur:
        print(f"Saisie invalide ({m!r}, {p!r}) :", erreur)
    except ZeroDivisionError:
        print(f"Poids nul pour {m} F : calcul impossible")
```

```
2500 F pour 2 kg : 1250 F/kg
Poids nul pour 3000 F : calcul impossible
Saisie invalide ('abc', '1') : invalid literal for int() with base 10: 'abc'
Saisie invalide ('1800', '') : could not convert string to float: ''
```

Morceau	Ce qu'il veut dire
Deux blocs `except`	Python essaie les `except` **dans l'ordre** et exécute le **premier** dont le type correspond
`except ValueError as erreur:`	`as erreur` range l'exception attrapée dans la variable `erreur`. L'afficher donne son **message**
`{m!r}`	Dans une f-string, `!r` affiche la valeur « comme dans le code », avec ses guillemets : on voit que `p` est un texte **vide** `''`
La boucle continue	Chaque tour a son propre `try` : une saisie invalide n'empêche pas de traiter les suivantes

Pour traiter plusieurs types **de la même façon**, on les regroupe dans un tuple : `except (ValueError, TypeError):`.

3.3 `else` et `finally`

Un `try` peut avoir deux blocs de plus. Créez `essai3.py` :

```python
def lire_poids(texte):
    try:
        poids = float(texte)
    except ValueError:
        print("  except : saisie non numérique")
        return None
    else:
        print("  else : conversion réussie")
        return poids
    finally:
        print("  finally : toujours exécuté")


for t in ["2.5", "deux"]:
    print("Saisie :", t)
    print("Résultat :", lire_poids(t))
```

```
Saisie : 2.5
  else : conversion réussie
  finally : toujours exécuté
Résultat : 2.5
Saisie : deux
  except : saisie non numérique
  finally : toujours exécuté
Résultat : None
```

Bloc	Quand est-il exécuté ?	A quoi sert-il ?
`try`	Toujours, jusqu'à la première exception	Le code qui peut échouer, et **seulement** lui
`except`	Seulement si une exception du type indiqué se produit dans le `try`	Réagir à l'erreur
`else`	Seulement si le `try` s'est terminé **sans** exception	La suite normale : garder dans le `try` uniquement la ligne risquée
`finally`	**Toujours**, avec ou sans exception, **même** si un `return` a déjà été rencontré	Libérer une ressource, fermer un fichier, afficher un bilan

Remarquez : le `finally` s'exécute **avant** que la fonction ne rende sa valeur, alors que le `return` est écrit dans le `else` ou le `except`.

3.4 Ce qu'il ne faut pas faire

Créez `essai11.py` :

```python
try:
    x = int("12")
except:
    print("erreur")
try:
    print(10 / 0)
except Exception:
    print("Une erreur, mais laquelle ?")
try:
    print(10 / 0)
except Exception as e:
    print("Erreur :", type(e).__name__, "-", e)
```

```
Une erreur, mais laquelle ?
Erreur : ZeroDivisionError - division by zero
```

Pratique	Pourquoi c'est dangereux	A la place
`except:` tout seul	Il attrape **tout**, y compris les fautes de frappe de votre code et le `Ctrl+C` de l'utilisateur. Vous cachez vos propres bugs	Nommez l'exception attendue : `except ValueError:`
`except Exception:` sans rien afficher	Le programme continue, mais personne ne sait ce qui s'est passé	Affichez au moins le type et le message (`type(e).__name__` , `e`), ou ne l'utilisez qu'en dernier recours, tout en haut du programme
Un `try` qui entoure 30 lignes	On ne sait plus quelle ligne a échoué	Un `try` court, autour de la ligne risquée, et `else` pour la suite

> 🔵 **Standard : deux philosophies** Python préfère « essayer, puis gérer l'échec » (en anglais *EAFP*, *easier to ask for forgiveness than permission*) plutôt que « tout vérifier avant » (*LBYL*, *look before you leap*). Par exemple, plutôt que de vérifier que chaque caractère de `texte` est un chiffre avant `int(texte)` , on essaie `int(texte)` et on attrape la `ValueError` . C'est plus court, et plus sûr : vos vérifications oublieraient le signe `-` , les espaces, les grands nombres...

4. Redemander tant que la saisie est invalide

La situation la plus fréquente : redemander une valeur jusqu'à ce qu'elle soit correcte. Créez `essai4.py` :

```python
def lire_entier(question, minimum, maximum):
    while True:
        texte = input(question)
        try:
            valeur = int(texte)
        except ValueError:
            print(f"  « {texte} » n'est pas un nombre entier.")
            continue
        if minimum <= valeur <= maximum:
            return valeur
        print(f"  Entrez un nombre entre {minimum} et {maximum}.")


nb = lire_entier("Nombre de colis (1 à 20) : ", 1, 20)
print("Colis à livrer :", nb)
```

Tapez `beaucoup`, puis `35`, puis `12` :

```
Nombre de colis (1 à 20) : beaucoup
  « beaucoup » n'est pas un nombre entier.
Nombre de colis (1 à 20) : 35
  Entrez un nombre entre 1 et 20.
Nombre de colis (1 à 20) : 12
Colis à livrer : 12
```

Morceau	Ce qu'il veut dire
`while True:`	Une boucle « infinie » : elle ne s'arrête que par un `return` (ou un `break`)
`continue`	Dans le `except` : on passe directement au tour suivant, sans tester les bornes d'une valeur qui n'existe pas
`if minimum <= valeur <= maximum:`	Python permet d'enchaîner les comparaisons : « `valeur` entre `minimum` et `maximum` »
`return valeur`	La seule sortie de la boucle : la valeur est forcément valide

Une fonction comme `lire_entier` s'écrit **une fois** et sert partout : c'est la base de l'atelier.

5. Lever une exception : `raise`

5.1 Signaler un problème depuis une fonction

Jusqu'ici, c'est Python qui levait les exceptions. Vos propres fonctions peuvent aussi en **lever**, quand on leur donne des valeurs absurdes. Créez `essai5.py` :

```python
TARIFS = {"Médina": 1000, "Plateau": 1500, "Yoff": 2000}


def frais_livraison(quartier, poids):
    if poids <= 0:
        raise ValueError(f"poids invalide : {poids}")
    if quartier not in TARIFS:
        raise KeyError(quartier)
    frais = TARIFS[quartier]
    if poids > 5:
        frais += 500
    return frais


print(frais_livraison("Plateau", 2))
print(frais_livraison("Yoff", 7.5))
for q, p in [("Médina", -1), ("Pikine", 3)]:
    try:
        print(frais_livraison(q, p))
    except ValueError as e:
        print("Erreur de valeur :", e)
    except KeyError as e:
        print("Quartier non desservi :", e)
```

```
1500
2500
Erreur de valeur : poids invalide : -1
Quartier non desservi : 'Pikine'
```

Morceau	Ce qu'il veut dire
`raise ValueError(f"poids invalide : {poids}")`	**Lève** une exception de type `ValueError`, avec un message. La fonction s'arrête immédiatement, comme avec un `return`, mais sans renvoyer de valeur
Pourquoi pas `return -1` ?	Un `-1` peut passer inaperçu et finir dans un total. Une exception ne peut pas être ignorée : soit quelqu'un l'attrape, soit le programme s'arrête et le dit
Qui choisit quoi faire ?	La fonction **signale** le problème ; c'est **celui qui l'appelle** qui décide de la réaction (afficher, redemander, ignorer la ligne)

Choisissez le type qui décrit le problème : `ValueError` pour une valeur incorrecte, `TypeError` pour un mauvais type, `KeyError` pour une clé absente...

5.2 L'exception remonte les appels

Créez `essai6.py` :

```python
def convertir(texte):
    return int(texte)


def total(lignes):
    somme = 0
    for ligne in lignes:
        somme += convertir(ligne)
    return somme


def programme():
    try:
        print("Total :", total(["2500", "1500", "mille"]))
    except ValueError as e:
        print("Erreur attrapée dans programme :", e)


programme()
```

```
Erreur attrapée dans programme : invalid literal for int() with base 10: 'mille'
```

L'exception est levée dans `convertir`, qui ne l'attrape pas ; elle **remonte** dans `total`, qui ne l'attrape pas non plus ; elle remonte dans `programme`, qui l'attrape. Si personne ne l'attrape, elle remonte jusqu'en haut et le programme s'arrête avec un traceback.

```
  programme()   ◄── l'exception est attrapée ici (try / except)
     │ appelle                        ▲
     ▼                                │ remonte
  total(...)                          │
     │ appelle                        │ remonte
     ▼                                │
  convertir("mille")  ── lève ValueError
```

C'est pour cela qu'on peut écrire des fonctions simples, qui ne gèrent pas les erreurs, et les attraper **à l'endroit où l'on sait quoi faire**.

6. La hiérarchie des exceptions

Les exceptions sont rangées en **familles**. Créez `essai7.py` :

```python
donnees = {"Modou": [2500, 3000], "Astou": []}
for cle in ["Modou", "Astou", "Ibou"]:
    try:
        montants = donnees[cle]
        print(cle, "premier montant :", montants[0])
    except LookupError as e:
        print(cle, ":", type(e).__name__, e)
print(ZeroDivisionError.__mro__)
print(KeyError.__mro__)
print(FileNotFoundError.__mro__)
```

```
Modou premier montant : 2500
Astou : IndexError list index out of range
Ibou : KeyError 'Ibou'
(<class 'ZeroDivisionError'>, <class 'ArithmeticError'>, <class 'Exception'>, <class
'BaseException'>, <class 'object'>)
(<class 'KeyError'>, <class 'LookupError'>, <class 'Exception'>, <class 'BaseException'>, <class
'object'>)
(<class 'FileNotFoundError'>, <class 'OSError'>, <class 'Exception'>, <class 'BaseException'>,
<class 'object'>)
```

Un seul `except LookupError` a attrapé **deux** exceptions différentes : `IndexError` (Astou n'a aucun montant) et `KeyError` (Ibou n'est pas dans le dictionnaire). Les lignes `__mro__` montrent la « famille » de chaque exception, de la plus précise à la plus générale :

```
BaseException
 ├── KeyboardInterrupt          (Ctrl+C)
 └── Exception                  (toutes les erreurs « normales »)
      ├── ArithmeticError
      │    └── ZeroDivisionError
      ├── LookupError
      │    ├── IndexError
      │    └── KeyError
      ├── OSError
      │    └── FileNotFoundError
      ├── ValueError
      └── TypeError
```

Règle : un `except` attrape le type nommé **et toute sa descendance**. Conséquence : quand on écrit plusieurs `except`, on met **les plus précis en premier**. Si `except Exception:` est en premier, les `except` suivants ne servent jamais.

> 🟢 **Débutant : et `__mro__`, c'est quoi ?** Les exceptions sont des **classes**, et les familles d'exceptions sont un exemple d'**héritage** : `KeyError` « hérite » de `LookupError`. Vous apprendrez à écrire des classes et à utiliser l'héritage au chapitre 7. Pour l'instant, retenez l'arbre.

7. Vos propres exceptions

🔵 **Standard : une exception personnalisée** Pour un programme important, on crée ses propres types d'exceptions, qui disent exactement ce qui ne va pas. Il suffit de deux lignes, à recopier telles quelles pour l'instant (les classes seront expliquées au chapitre 7). Créez `essai8.py` :

```
class MontantInvalideError(ValueError):
    pass


def verifier_montant(montant):
    if montant < 500:
        raise MontantInvalideError(f"{montant} F : le minimum est 500 F")
    if montant > 500000:
        raise MontantInvalideError(f"{montant} F : le maximum est 500 000 F")
    return montant


for m in [2500, 200, 900000]:
    try:
        print("Accepté :", verifier_montant(m))
    except MontantInvalideError as e:
        print("Refusé :", e)
print(isinstance(MontantInvalideError("x"), ValueError))
```

Résultat : `Accepté : 2500`, `Refusé : 200 F : le minimum est 500 F`, `Refusé : 900000 F : le maximum est 500 000 F`, puis `True`.

Morceau	Ce qu'il veut dire
`class MontantInvalideError(ValueError):`	Crée un nouveau type d'exception, de la **famille** de `ValueError`. Par convention, le nom se termine par `Error`
`pass`	« rien de plus » : le nouveau type se comporte exactement comme `ValueError` ; seul son nom change
`isinstance(..., ValueError)` vaut `True`	Une `MontantInvalideError` **est aussi** une `ValueError` : un `except ValueError` l'attraperait

> 🟣 **Expert : `assert` et le chaînage des exceptions** `assert condition, "message"` lève une `AssertionError` si la condition est fausse. Il sert à vérifier **vos propres hypothèses de programmeur** (« ce taux est forcément entre 0 et 1 »), pas les saisies de l'utilisateur : Python peut désactiver les `assert` (option `-O`). Avec `assert 0 <= taux <= 1, f"taux hors de [0, 1] : {taux}"` et un taux de 10 : `AssertionError: taux hors de [0, 1] : 10`.
>
> Pour transformer une exception technique en exception plus parlante sans perdre la cause : `raise RuntimeError(f"ligne de registre illisible : {texte!r}") from e`. Le traceback affiche alors les deux, reliés par *The above exception was the direct cause of the following exception* (« l'exception ci-dessus est la cause directe de la suivante »).

8. Les bonnes pratiques, en résumé

1. **Un `try` court**, autour de la ligne qui peut échouer ; la suite normale dans `else`.
2. **Nommez l'exception attendue** ; jamais de `except:` seul.
3. **Les plus précises d'abord** quand il y a plusieurs `except`.
4. **Un message utile** pour l'utilisateur : ce qui ne va pas **et** ce qu'il doit faire (« Utilisez le format JJ/MM/AAAA »).
5. **Les fonctions de calcul lèvent** (`raise`), **le programme qui dialogue avec l'utilisateur attrape** (`try` / `except`).
6. `finally` pour ce qui doit toujours être fait (fermer, faire le bilan).

Exercices

Créez un dossier `exercices-ch1`, avec un fichier par exercice.

- ★ **Exercice 1 : convertir une note.** Écrivez la fonction `convertir_note(texte)` qui accepte une note sur 20 tapée avec un point **ou** une virgule, avec des espaces autour, et renvoie un `float` ; elle renvoie `None` en affichant un message si le texte n'est pas un nombre ou si la note est hors de 0 à 20. Tests : `"15,5"` donne 15.5 ; `"abc"`, `"21"` et `"-0.5"` donnent `None` ; `" 12 "` donne 12.0. (Sections 3.1 et 4.)
- ★ **Exercice 2 : prédire avant d'exécuter.** Sans lancer le programme, écrivez sur une feuille les quatre lignes affichées. Puis vérifiez, et expliquez chaque erreur de prédiction. Attention à la dernière : que vaut `100 // -3` ? (Section 3.3.)

```python
def tester(valeur):
    try:
        resultat = 100 // int(valeur)
    except ValueError:
        print("A", end=" ")
        resultat = -1
    except ZeroDivisionError:
        print("B", end=" ")
        resultat = 0
    else:
        print("C", end=" ")
    finally:
        print("D", end=" ")
    return resultat


print(tester("7"))
print(tester("0"))
print(tester("sept"))
print(tester("-3"))
```

- ★★ **Exercice 3 : une moyenne malgré les valeurs sales.** Écrivez `moyenne_valide(valeurs)` qui calcule la moyenne des valeurs **convertibles** en nombre d'une liste, et compte les valeurs rejetées. Elle renvoie un tuple `(moyenne, nombre_de_rejets)`, avec `None` comme moyenne s'il n'y a aucune valeur valide. Avec `["12", "15.5", "abc", "", "18", None]` : `(15.166666666666666, 3)`. Quelle exception lève `float(None)` ? Et `float("")` ? (Sections 2.3 et 3.2.)
- ★★ **Exercice 4 : l'indice de masse corporelle.** Écrivez `calculer_imc(poids_kg, taille_m)` (poids divisé par la taille au carré) qui **lève** une `ValueError` avec un message clair si le poids n'est pas positif, ou si la taille n'est pas entre 0,5 et 2,5 m (un utilisateur qui tape 175 pense en centimètres). Testez avec (70, 1.75) : 22.9 ; (70, 175) et (-2, 1.6) : erreurs ; (54, 1.62) : 20.6. (Section 5.)
- ★★★ **Exercice 5 : les réservations du car.** Créez l'exception `PlacesInsuffisantesError`. La fonction `reserver(places_libres, demande)` renvoie le nombre de places restantes ; elle lève une `ValueError` si la demande n'est pas positive, et une `PlacesInsuffisantesError` s'il n'y a pas assez de places. Traitez la liste de demandes `[4, 0, 6, 8, 5]` pour un car de 15 places, en affichant pour chacune « acceptée », « ignorée » ou « refusée » avec le message, puis le bilan : 3 réservations acceptées, 0 place libre. (Sections 5 et 7.)

Atelier 1 : la saisie fiable des livraisons

Le travail principal du chapitre : vous écrivez le programme de saisie des livraisons de Yonnee Express, qui **ne plante jamais**, quelle que soit la réponse de l'utilisateur. L'énoncé de l'atelier vous est fourni séparément.

A retenir

- Une **erreur de syntaxe** empêche le programme de démarrer ; une **exception** survient pendant l'exécution et peut être interceptée.

- Un traceback se lit **par la fin** : la dernière ligne dit quoi (type et message), la ligne `File ...` au-dessus dit où.
- `try` / `except Type:` intercepte une exception ; `as e` donne son message ; plusieurs `except` sont testés dans l'ordre.
- `else` : si tout s'est bien passé ; `finally` : toujours, même après un `return`.
- `while True` + `try` + `continue` + `return` : le modèle pour redemander une saisie.
- `raise` : une fonction signale un problème ; l'exception **remonte** les appels jusqu'à un `except`.
- Les exceptions forment une **hiérarchie** : un `except` attrape aussi les sous-types ; les plus précis d'abord ; jamais de `except:` seul.

Erreurs fréquentes

Message ou symptôme	Ce que ça veut dire	Solution
`SyntaxError: expected ':'`	`:` oublié à la fin d'un `if`, `for`, `while`, `def`, `try`, `except`	Ajouter le `:`
`IndentationError: expected an indented block ...`	Bloc non décalé	Décaler de 4 espaces le corps du bloc
`ValueError: invalid literal for int() with base 10: '...'`	`int()` sur un texte qui n'est pas un entier (lettres, virgule, espaces au milieu)	`try` / `except ValueError` ; nettoyer le texte (`strip()`, `replace`)
`ValueError: could not convert string to float: ''`	`float()` sur un texte vide	Redemander la saisie
`TypeError: can only concatenate str (not "int") to str`	`+` entre un texte et un nombre	f-string : `f"Montant : {m}"`
`KeyError: '...'`	Clé absente d'un dictionnaire	Tester `if cle in dico`, ou `dico.get(cle)`
`NameError: name '...' is not defined`	Faute de frappe, ou variable utilisée avant d'être créée	Lire la suggestion `Did you mean`
Le programme plante malgré un `try`	L'`except` ne nomme pas le bon type, ou la ligne qui échoue est hors du `try`	Lire le type dans le traceback ; déplacer la ligne dans le `try`
Une erreur passe inaperçue	`except:` seul ou `except Exception: pass`	Nommer l'exception, afficher le message
Un `except` n'est jamais exécuté	Un `except` plus général est placé avant	Mettre les exceptions les plus précises en premier

Pour aller plus loin

- Le tutoriel officiel de Python, en français, chapitre « Erreurs et exceptions » : https://docs.python.org/fr/3/tutorial/errors.html
- La liste et la hiérarchie des exceptions intégrées : https://docs.python.org/fr/3/library/exceptions.html
- Déboguer un programme Python dans VS Code (en anglais), pour suivre une exception pas à pas : https://code.visualstudio.com/docs/python/debugging